

Bowling Score Detection from Video

Using Computer Vision and Deep Learning

DSAI 352 — Computer Vision | Spring 2026

Final Project Technical Report

Rana Ahmed — 202201899

Toka Mokhtar — 202201920

1. Project Overview

This project implements a computer vision pipeline that processes a recorded bowling video and produces an annotated output video together with a final score. The system detects when each pin falls, marks it visually on the output frames, records the timestamp at which it fell, and reports the total number of pins knocked down over the duration of the video.

The setup uses a simplified bowling scenario with six pins arranged on the ground (two per color: blue, red, yellow). One team member records on a mobile phone while the other rolls a bowling ball toward the pins. Multiple throws are performed within a single approximately 30-second recording.

2. Pipeline Architecture

The pipeline accepts a single mobile-recorded video as input and produces an annotated video as output. The pipeline consists of seven stages:

1. Video ingestion and preprocessing (4K → 1080p resize)
2. Frame extraction for dataset creation
3. Dataset annotation using Roboflow
4. Model fine-tuning (YOLOv8s on custom dataset)
5. Detection and tracking using ByteTrack
6. Fall detection using movement-based analysis
7. Annotated video rendering with OpenCV

3. Dataset

3.1 Data Collection

The dataset was collected by extracting one frame per second from the bowling video, yielding 31 labeled images. The video was recorded on a mobile phone at 4K resolution (3840×2160) and 60 FPS. Before frame extraction, the video was downscaled to 1080p (1920×1080) to reduce computational cost.

3.2 Annotation

Frames were annotated using Roboflow with bounding boxes drawn around each pin. Three class labels were used corresponding to pin color: blue, red, and yellow. The dataset was split as follows:

Split	Images	Percentage
Train	23	74%
Validation	5	16%
Test	3	10%

4. Model Architecture and Training

4.1 Model Choice

YOLOv8s (small variant) was selected as the base model. YOLOv8 is a state-of-the-art single-stage object detector that provides an effective balance between detection speed and accuracy. The pretrained weights (trained on the COCO dataset) provide a strong visual backbone, and fine-tuning adapts the detection head to recognize our specific pin classes.

This approach satisfies the project modeling requirement: a pretrained model is fine-tuned on a custom dataset rather than using an off-the-shelf model without any training step.

4.2 Training Configuration

Parameter	Value
Base model	yolov8s.pt (pretrained on COCO)
Epochs	50 (early stopping at 22)
Image size	640 × 640
Batch size	8
Optimizer	AdamW (lr=0.001429)
Early stopping patience	10 epochs
Framework	Ultralytics YOLOv8
Training environment	Google Colab (CPU)

4.3 Training Results

Training converged at epoch 22 (early stopping triggered after no improvement for 10 epochs). The best model was saved at epoch 12. Final validation metrics on the best checkpoint:

Class	Precision	Recall	mAP50	mAP50-95
all	0.994	0.969	0.995	0.975
blue	0.992	1.000	0.995	0.979

red	1.000	0.907	0.995	0.963
yellow	0.992	1.000	0.995	0.982

The model achieved near-perfect detection performance (mAP50 = 0.995) on the validation set across all three pin color classes, indicating that fine-tuning on a small custom dataset (31 images) was sufficient for reliable pin detection.

5. Multi-Object Tracking

ByteTrack was used as the tracking algorithm, integrated directly within the Ultralytics YOLOv8 inference pipeline. ByteTrack assigns stable track IDs to detected objects across frames by associating detections with existing tracks using IoU-based matching.

Since the game uses exactly 6 pins (2 per color), post-processing was applied to keep only the first 2 track IDs per color class based on first appearance. This eliminates spurious track IDs that arise when pins temporarily disappear and reappear between frames.

6. Fall Detection

6.1 Why Not Aspect Ratio?

An initial approach based on bounding-box aspect ratio (height/width) was investigated. The reasoning was that a standing pin would have a tall narrow box (aspect ratio > 1) while a fallen pin would have a wide flat box (aspect ratio < 1). However, the camera was positioned overhead (top-down view), meaning all pins appear roughly circular or oval regardless of whether they are standing or fallen. The aspect ratio signal was therefore unreliable and this approach was discarded.

6.2 Movement-Based Fall Detection

The adopted approach tracks the center-point displacement of each pin across consecutive frames. A pin fall produces a characteristic pattern in the displacement signal: a sudden spike (the pin is knocked and moves) followed by a period of stability (the pin has come to rest on the ground).

The fall detection algorithm iterates over the displacement signal for each pin and searches for the first frame where two conditions are met simultaneously:

- The displacement exceeds a movement threshold (MOVEMENT_THRESHOLD = 10 pixels), indicating a sudden positional change.
- The average displacement over the following 10 frames is below a stability threshold (STABLE_THRESHOLD = 8 pixels), confirming the pin has stopped moving.

When both conditions are satisfied, the pin is marked as fallen and the frame index is recorded. The corresponding timestamp in seconds is derived by dividing the frame index by the video frame rate (59.9 FPS).

6.3 Fall Detection Results

Pin ID	Color	Status	Fell at (sec)	Notes
--------	-------	--------	---------------	-------

1	blue	Fallen	14.73s	Detected correctly
2	red	Standing	-	Did not fall in video
3	yellow	Fallen	24.27s	Detected correctly
4	yellow	Fallen	3.14s	Detected correctly
5	blue	Fallen	27.46s	Detected correctly
6	red	Fallen	1.88s	Detected correctly

5 out of 6 pins were correctly identified as fallen. Pin 2 (red) remained standing throughout the video and was correctly classified as standing.

7. Output Video Rendering

The annotated output video is rendered using OpenCV and includes the following visual elements:

- Color-coded bounding box on each detected pin (blue, red, or yellow matching the pin color).
- Red X marker drawn over each pin from the frame it is detected as fallen onward.
- Timestamp label next to each fallen pin indicating the second at which the fall occurred (e.g., "PIN 1 fell @ 14.7s").
- HUD overlay in the top-left corner showing the current video time and the running count of pins knocked down (e.g., "Pins Down: 3/6").
- GAME OVER summary displayed in the final 3 seconds of the video, showing total pins down and total game duration.

8. Limitations and Known Failure Modes

- Small dataset: only 31 labeled images were used for training. A larger dataset would improve generalization to different lighting conditions, backgrounds, and pin arrangements.
- Track ID instability: when a pin is briefly occluded or leaves the frame, ByteTrack may assign a new track ID upon reappearance. The post-processing step (keeping only the first 2 IDs per color) mitigates this but may fail if the original IDs are reassigned to noise detections.
- Top-down camera angle: the overhead view eliminates the aspect-ratio fall signal that would be available from a side-on or angled view. The movement-based approach is more robust but relies on the pin physically moving when it falls, which may not always produce a large enough displacement at high framerates.
- Fixed pin count assumption: the pipeline is hard-coded to expect exactly 6 pins (2 per color). A different pin count or color scheme would require adjusting the valid ID selection logic.
- No ball detection: the bowling ball is not tracked. Ball trajectory information could be used to improve fall detection timing and reduce false positives.

9. Tools and Libraries

Tool / Library	Version	Purpose
Python	3.12	Primary programming language
Ultralytics YOLOv8	8.4.48	Detection and tracking
OpenCV	4.10.0	Video I/O and rendering
ByteTrack	(via Ultralytics)	Multi-object tracking
Roboflow	1.3.8	Dataset annotation and export
Google Colab	-	Training environment
NumPy	2.0.2	Numerical operations
Matplotlib	3.10.0	Displacement visualization

10. Conclusion

This project successfully demonstrates a complete computer vision pipeline for automated bowling score detection from video. The system fine-tunes a pretrained YOLOv8s model on a small custom dataset of 31 labeled images, achieving near-perfect detection accuracy (mAP50 = 0.995). A novel movement-based fall detection approach was developed to handle the top-down camera angle, where traditional aspect-ratio methods are ineffective. The pipeline correctly identified 5 out of 6 pins and their exact timestamps of falling, and produced a fully annotated output video meeting all project requirements.